Project 1

CSI 4900/5900 Agentic AI

Due by 11/03/2026 (Tuesday) 11:59:59pm

Summary

Design, build, and evaluate a multi-agent AI coach that generates personalized suggestions to help college students manage everyday stress. Build your system using existing large language models (LLMs); training or fine-tuning a model is not required.

Use historical smartphone sensing and self-report data from the StudentLife dataset to simulate a student's changing circumstances over time. Your system should analyze available activity, sleep, and social interaction information, decide whether a supportive message is appropriate, and generate a suggestion or ask a clarifying question when needed. Suggestions may include a study break, a manageable activity, or a relevant campus resource.

The coach should remember and use the student's goals, preferences, previous suggestions, and feedback across interactions. Use clearly labeled simulated profiles and interactions for information not available in the dataset.

This project should be completed in groups of 1-2 people. You need to find your group member by yourself.
Sign up your group in the Google sheet: CSI 4900/5900 - Fall 2026 - Project 1 - Group
You still need to sign up even if you do it individually.

Grading will be based on the code, report, and presentation.

Description

1. Dataset: You can download it from https://www.kaggle.com/datasets/dartweichen/student-life

StudentLife is the first study that uses passive and automatic sensing data from the phones of a class of 48 Dartmouth students over a 10 week term to assess their mental health (e.g., depression, loneliness, stress), academic performance (grades across all their classes, term GPA and cumulative GPA) and behavioral trends (e.g., how stress, sleep, visits to the gym, etc. change in response to college workload -- i.e., assignments, midterms, finals -- as the term progresses).

The StudentLife dataset does not include students' personal goals or coaching preferences. You may create these profiles manually or generate them randomly using code, based on your system design.

About how to use the data, you can refer to: StudentLife_Quick_Start.pdf

2. You may choose your agents' roles and how they interact. Your implementation and experiments must demonstrate the following design patterns. **(40 pts, 8 pts each)**

1. **Reflection**
2. **Tool Use**
3. **Planning**
4. **Multi-agent System**
5. **Memory and Skills**

Implement the relevant components so that they can be enabled, disabled, or varied for the ablation study. You may omit a component from your final configuration if your experiments justify that choice; however, the final system must retain at least two interacting agents.

3. Output Format: A test case represents one participant at a specified decision time, together with the observations, profile information, and interaction history available at that time.**(5 pts)**

Save your system's outputs in a **JSONL file**, with one JSON object per line for each test case. You may choose the fields and structure that best fit your system; you do not need to match the example below exactly. Briefly document your output fields in the README. The example is for illustration only and does not represent actual StudentLife results.

```
{
 "case_id": "case_001",
 "participant_id": "u12",
 "decision_time": "2013-04-15T09:00:00-04:00",
 "decision": "suggest",
 "evidence": [
  {
   "source": "simulated_profile",
   "detail": "The student prefers quiet individual activities."
  }
 ],
 "final_message": "Would you like to take a short, quiet break?",
 "trace": {
  "llm_calls": 3,
  "total_tokens": 2100,
  "latency_s": 4.2
 }
```

}

4. Functional Requirements and Code Evaluation: Your submission **must** include a documented command or entry point that runs your evaluation cases and saves the final outputs and execution logs. **(15 pts, 5 pts each)**
We will run your code following the README and inspect the outputs, logs, and relevant implementation. Checks will cover:

1. Successful execution: The system runs successfully, saves outputs, and reports any failed cases.
2. Functionality: The implementation demonstrates the design patterns described in Section 2.
3. Appropriate behavior: The system generates suggestions or messages appropriate to the student's available information.

Include reproducible cases covering typical inputs, missing data, and preference updates across interactions. Label simulated information and document case selection, model settings, dependencies, and memory reset instructions in the README.

5. Evaluation Metrics & Visualizations: Report the following metrics via figures or tables for your final system. **(20 pts, 5 pts each)**

1) **LLM judge on message quality**: Use a separate LLM judge to score each non-null final message from 1–5 on context awareness, goal alignment, and clarity. Provide the same available context and report the evaluation prompt and rubric.
2) **Human judge on message quality:** Independently rate at least 10 final messages using the same rubric before viewing the judge's scores.
3) **Difference between human and LLM judge scores**: Report the mean absolute difference between human and LLm judge scores and discuss notable disagreements.
4) **Efficiency:** Report end-to-end latency, LLM calls, and total tokens per case. A **case** is one participant at a specified decision time, with the information available then.
   End-to-end latency is the time from starting that case to producing its final output, including tool calls and all model steps, but excluding judge evaluation.

6. **Ablation Study**:

Conduct controlled comparisons to examine how design choices affect decision appropriateness, message quality, execution success, and efficiency. **(10 pts total)**

Include at least following comparisons, you can include more as needed:

1. **Multi-agent design:** Compare your full system with a single-agent baseline given the same observations, profile/history, and available tools.
2. **Reflection:** Compare the system with and without reflection.
3. **Planning:** Compare the system with and without planning.
4. **Reflection depth:** Compare maximum reflection budgets of 1, 2, and 3 (or more) review-and-revision cycles.
5. **Planning depth:** Compare maximum budgets of 1, 2, and 3 (or more) planning rounds.

6. **Model choice:** Compare at least two different LLMs under the same system configuration.

Ablation study is a crucial element in research papers, providing valuable insights into the contribution of each component of a model or system. About how to conduct an ablation study, refer to the ablation sections of agent papers published in top AI conferences, e.g., NeurIPS, ICML, ICLR, ACL, EMNLP.

Report (5 points)
Presentation (5 points)

Submission

Project submission includes two steps.

Step 1: You need to submit your implementation on Moodle by 11/03/2026 11:59:59pm. The submission link will be closed immediately after the deadline. Your submission should be a single zip file that is named by the last names of your team members. You may use this naming structure: <groupXX_lastname1_lastname2>.zip.

The zip file should contain the following:

- A folder containing all your code.
- A simple README file that describes the structure of your code and how to run it.
- A PDF report. Use one of the provided templates to prepare your report.
    - Google Doc template: CSI 4900/5900 — Project Report
    - Overleaf template: https://www.overleaf.com/read/pmvkxzvwdsrp#c853d1
    - You will get bonus points if you write your report following ICML 2026 LaTeX style and your formatting, tables, and figures are well-organized and visually appealing. ICML 2026 LaTeX template is at: https://icml.cc/Conferences/2026/CallForPapers or I downloaded for you: https://drive.google.com/file/d/1CD2RdRLsTM15OF44tpOa_tYueYnO6I-x/view?usp=drive_link
- Evaluation case definitions, including participant IDs, decision times, and clearly labeled simulated profiles or feedback.
- Final JSONL outputs, execution logs, and scripts/configurations needed to reproduce the evaluation and ablation results.

Do not include the raw dataset, model weights, virtual environments, or generated caches. Include the required PDF report and the code and evaluation materials listed above. Only one submission is needed per group.
Failure to follow the above instructions to prepare your submission will cause a penalty to your grade.

Step 2: You need to present your project in class, 11/05/2026. You have about 5 minutes for the presentation. The presentation duration may be adjusted before one week of the deadline according to the number of groups.

Policies

1) Late submissions: refer to the late policy in the syllabus.
2) Each group needs to work independently on this exercise. We encourage high-level discussions among groups to help each other understand the concepts and principles. However, code-level discussion is prohibited and plagiarism will directly lead to failure of this course.